

The DSA/LeetCode Survival Guide — Lessons From My First 29 Days

A guide for people just starting their DSA or LeetCode journey, built from real mistakes I made (and kept a log of) while learning.

Why This Guide Exists

When I started solving LeetCode problems, I thought the hard part would be *finding* the solution. It wasn't. The hard part was training my brain to break a problem into small logical steps and then turning those steps into working code. Knowing a concept and being able to implement it correctly are two completely different skills — and almost nobody tells you that when you start.

This guide is everything I wish someone had told me on Day 1.

1. Get Your Mindset Right Before You Touch Code

This matters more than any algorithm.

- **Progress isn't always visible.** Some days you'll solve 5 problems and feel unstoppable. Other days you'll spend the whole session stuck on one problem, confused, and walk away having "solved" nothing — but you'll understand something you didn't

before. Both are real progress. The invisible days (confusion, mistakes, slow persistence) count just as much as the productive-looking ones.

- **Stop comparing your speed to other people's.** Someone solving a problem faster than you doesn't mean they're learning it better. Speed comes later, automatically, once understanding is solid. Chasing speed early just makes you skip understanding.
 - **Notice the shift from "learning algorithms" to "learning how to express algorithms in code."** Early on, your bottleneck is usually logic. Later, your bottleneck becomes syntax and translation — turning a correct idea into correct code. Realizing which one is currently holding you back tells you what to actually practice.
 - **A pattern becomes easy when you understand *why* it works, not when you memorize its template.** Memorized templates fall apart the moment a problem is phrased slightly differently. Understood patterns adapt.
-

2. Don't Rush to Hard Problems

Easy problems aren't "beneath you" when you're starting out — they're where you build the foundation everything else sits on: array traversal, index handling, return types, basic conditionals. Rushing to Medium/Hard problems before these are automatic just means you're fighting the syntax *and* the logic at the same time, and you won't be able to tell which one is actually failing you.

Build fluency on Easy problems first. Speed and confidence on Hard problems are a *side effect* of that fluency, not something you can shortcut into.

3. Understand the Problem Before You Write a Single Line

This is probably the single most repeated lesson across weeks of practice: **rushing into code when the problem statement itself is still unclear is the #1 cause of wasted time.**

Before coding, always ask:

```
What exactly is the input?  
What exactly is the output?  
What does each important word in the problem actually mean?  
What data structure fits this naturally?  
What pattern does this problem resemble?
```

Pay close attention to small wording differences — they change the entire approach. For example:

```
"prefix"      → must start at the beginning  
"substring"   → can start anywhere, but must be continuous  
"subsequence" → can skip elements, order preserved
```

Misreading one of these words can send you down the wrong solution path for 30 minutes before you realize the bug isn't in your code — it's in your understanding.

A simple problem-solving loop that works:

1. Understand the problem statement.
2. Identify input and output clearly.
3. Think through a brute-force approach first (even if it's slow).
4. Write the logic in plain words/pseudocode before touching syntax.
5. Convert that logic into code.
6. Debug what breaks.
7. Write down what you learned.
8. Save any reusable pattern for later.

Steps 7 and 8 are the ones beginners skip — and they're the ones that compound over time.

4. Keep a Mistake Log — Seriously

Your mistakes are your personal roadmap. Not a metaphor — an actual, literal log you write down and revisit. The same handful of mistakes will keep showing up in disguise across many different problems. If you write them down once, you'll recognize them instantly the next ten times they appear. If you don't, you'll re-debug the exact same bug for months.

Some categories worth tracking, with real examples:

Syntax-level mistakes (language-specific, but the *pattern* generalizes to any language):

- Confusing a property vs. a method (e.g. `array.length` vs `string.length()` in Java) — always check whether you're dealing with an array or an object with methods.
- Comparing objects/strings with `==` instead of a proper equality check (`.equals()` in Java). `==` often compares references, not values.
- Confusing a single character type with a character array/string type.
- Wrong array declaration syntax (declaring size in the wrong place, forgetting `new`).
- Returning the wrong data type from a function — e.g. building a `char[]` when the function signature expects a `String`. You need an explicit conversion step; it won't happen automatically.
- Building your final data structure (like a `String`) too early, before you've finished filling in the data. Fill the raw structure completely first, convert once at the end.

Logic-level mistakes (these matter far more than syntax):

- Confusing "the whole array" with "an individual element" — very common in 2D array problems, where it's easy to accidentally operate on a full row when you meant a single cell, or vice versa.
- Moving a pointer (e.g. in sliding-window problems) without updating the state that pointer was tracking — like sliding the window forward but forgetting to remove the outgoing element's effect on your running count/sum.

- Knowing *what operation* to do but not knowing *when to stop* — e.g. writing correct division logic but getting the stopping/base condition wrong. The operation and the termination condition are two separate things to get right.
- Confusing "counting occurrences" with "counting the wrong thing entirely" — always re-read what the problem actually wants counted, not what seems intuitive.

How to log a mistake usefully (don't just write "I had a bug"):

```
Problem: [name/number]
What I thought would happen:
What actually happened:
Root cause (be specific — syntax? logic? misread problem?
The fix:
The general pattern to remember next time:
```

5. Build a Personal Pattern Library

As you solve problems, you'll notice the same reusable shapes of code keep coming back. Save them somewhere you'll actually revisit — a notes file, a repo, whatever. A few examples of the kind of thing worth saving (language-agnostic in spirit, shown in Java-style pseudocode):

Traversal

```
for (int i = 0; i < arr.length; i++) { }
```

Running/prefix accumulation — current answer depends on the previous value:

```
arr[i] = arr[i] + arr[i - 1];
```

Tracking a maximum (or minimum)

```
if (current > max) {  
    max = current;  
}
```

Two pointers — keep pointers separate and move them independently instead of reusing one index for two purposes.

Sliding window — expand the right edge, and when the window becomes invalid, shrink from the left *while correctly undoing* whatever state change that removed element caused.

Binary search isn't just for finding an element in a sorted array — it can search for an *answer* too (binary search on the answer space), and it can be adapted for conditions like "find the first true value" or peaks in mountain-shaped arrays. Recognize binary search by: sorted (or monotonic) structure + you're trying to minimize/maximize something or find a boundary.

Recursion, once it clicks, is really just:

1. Define a base case (when do you stop?)
2. Reduce the problem to a smaller version of itself.
3. Trust that the smaller version returns the correct answer.
4. Build the final answer from that trusted smaller result.

"Trust the smaller answer" is the step beginners struggle with most — you don't need to trace the entire call stack in your head to believe the recursion works.

The point of a pattern library isn't to memorize code snippets — it's that recognizing "oh, this is a two-pointer problem" or "this is binary-search-on-answer" turns a scary new problem into a variation of something you already know.

6. A Sane Topic Order for Beginners

Roughly the order that builds naturally on itself:

1. **Arrays & loops** — traversal, prefix sums, basic conditionals
2. **Strings** — traversal, comparison, character access/conversion
3. **ArrayLists / dynamic arrays & hashmaps** — frequency counting, grouping by key
4. **Two pointers**
5. **Sliding window**
6. **Binary search** (including binary search on the answer)
7. **Recursion**
8. **Linked lists**
9. **2D arrays / matrices** (diagonals, row vs. element confusion is very common here — go slow)
10. From there: stacks/queues, trees, graphs, dynamic programming

Don't feel behind if this takes weeks, not days. It's normal.

7. What To Do When You're Stuck or Frustrated

- If a "simple-looking" problem turns into a deep rabbit hole (this happens most with recursion and math-heavy problems), that's not a sign you're bad at this — it's usually a sign the problem is teaching you something bigger than itself.
 - When everything feels mixed up — syntax, logic, and the math of the problem all tangled together — stop trying to fix the code. Step back and re-explain the problem to yourself in plain words first.
 - It's fine to solve fewer problems on a hard day. One problem deeply understood beats five problems copy-pasted and forgotten.
 - Keep some kind of streak or log of days practiced — not for the guilt trip, but because looking back at week 1 vs. week 3 is the best motivation you'll get. You'll see it in your own words: what confused you a month ago will look obvious later.
-

8. Things Not To Do

- **Don't memorize solutions without understanding them.** It feels productive short-term and fails you completely on any problem phrased slightly differently.
 - **Don't skip writing down what you learned** after solving a problem. The 2 minutes it takes is what turns "I solved this once" into "I recognize this pattern forever."
 - **Don't treat every new problem as 100% new.** Actively ask "have I seen a shape like this before?" before diving in.
 - **Don't rush past the problem statement.** Misunderstanding the question is a bigger time-waster than any bug you'll write.
 - **Don't compare your pace to others' streaks or problem counts.** Different people start from different places.
-

Final Message

I started with confusion. I struggled with syntax, with indexes, with turning ideas into working code. But every mistake added one small piece of understanding.

The goal was never to solve thousands of problems as fast as possible. The goal is becoming a better problem solver, one problem at a time — and that only happens if you actually stop to notice what each mistake is teaching you.

Good luck. Log your mistakes. Understand before you code. And don't rush. 🐟